

Medium

Search



Be part of a better internet. [Get 20% off membership for a limited time](#)

Format LLM Output to Generate JSON



Abdullah Şamil Güser · Follow

3 min read · Jun 20, 2024



Listen



Share



More

My aim in this post is to list the current available methods that are used to generate JSON formatted outputs from LLMs.

I will update this post occasionally while I learn new methods.

All methods are available as a jupyter notebook under my [repository](#). So you can directly use the notebook itself if you want a quick start.

1. Use Grammar Rules to Force LLM to Output JSON

[Llama.cpp](#) uses [formal grammars](#) to constrain model output to generate json formatted text. Grammars are defined in GBNF (GGML BNF) format.

Prerequisites

- [PyTorch](#)
- [huggingface-hub==0.23.0](#)

There are 2 different ways that we can apply this method.

1st Option : llama.cpp

Our first option is to use the [llama.cpp](#) library with a grammar file to generate the JSON output.

1. Clone the repository

```
git clone https://github.com/ggerganov/llama.cpp.git
```

2. Change directory under *llama.cpp* and build the project.

```
make -j
```

3. Download *mistral-7b-instruct-v0.1.Q8_0.gguf* under *./models* directory.

```
huggingface-cli download TheBloke/Mistral-7B-Instruct-v0.1-GGUF mistral-7b-instruct-v0.1.Q8_0.gguf --local-dir .
```

4. Create a grammar file under `./grammars` directory. For example, let's say your json shema looks like this:

```
interface answer {
  id: number;
  name: string;
}
```

Then your grammar file (`answer.gbnf`) will look like this

```
root ::= answer
answer ::= "{" ws "\"id\":" ws number "," ws "\"name\":" ws string "}"
answerlist ::= "[" | "[" ws answer ("," ws answer)* "]"
string ::= "\"" ([^"]*) "\""
boolean ::= "true" | "false"
ws ::= [ \t\n]*
number ::= [0-9]+ "."? [0-9]*
stringlist ::= "[" ws string ("," ws string)* ws "]"
numberlist ::= "[" ws number ("," ws number)* ws "]"
```

You can use grammar.intrinsiclabs.ai to generate a grammar file for your custom schemas.

6. Now you can run below command under `llama.cpp` directory to generate JSON output.

```
./llama-cli -m ./models/mistral-7b-instruct-v0.1.Q8_0.gguf -n 256 --grammar-file grammars/answer.gbnf -p "Q: Nar
```

2nd Option (Preferred) : use llama-index

[llama-index](#) is a very cool Python package that can be used to replicate llama.cpp functionality in Python.

1. Install python library

```
pip install llama-index==0.10.46
pip install llama-index-embeddings-huggingface==0.2.2
pip install llama-index-llms-llama-cpp==0.1.4
```

2. Download `mistral-7b-instruct-v0.1.Q8_0.gguf`.

```
huggingface-cli download TheBloke/Mistral-7B-Instruct-v0.1-GGUF mistral-7b-instruct-v0.1.Q8_0.gguf --local-dir .
```

3. Define json schema

```
from llama_cpp.llama_grammar import LlamaGrammar
from pydantic import BaseModel

class Answer(BaseModel):
    id: int
```

```

name: str

class Answers(BaseModel):
    answers: list[Answer]

model_schema = str(Answers.model_json_schema()).replace("'", '"')

grammar = LlamaGrammar.from_json_schema(json_schema=model_schema)

```

4. Define LLama model

```

from llama_index.llms.llama_cpp import LlamaCPP
from llama_index.llms.llama_cpp.llama_utils import (
    messages_to_prompt,
    completion_to_prompt,
)

model_url = "./mistral-7b-instruct-v0.1.Q8_0.gguf"

llm = LlamaCPP(
    # You can pass in the URL to a GGML model to download it automatically
    # model_url=model_url,
    # optionally, you can set the path to a pre-downloaded model instead of model_url
    model_path=model_url,
    temperature=0.1,
    max_new_tokens=256,
    # llama2 has a context window of 4096 tokens, but we set it lower to allow for some wiggle room
    context_window=3900,
    # kwargs to pass to __call__()
    generate_kwargs={
        "grammar": grammar,
    },
    # kwargs to pass to __init__()
    # set to at least 1 to use GPU
    model_kwargs={"n_gpu_layers": 1},
    # transform inputs into Llama2 format
    messages_to_prompt=messages_to_prompt,
    completion_to_prompt=completion_to_prompt,
    verbose=True,
)

```

5. Run

```

response = llm.complete("Q: Name all of the planets in the solar system? A:")
print(response.text)

```

References

- [GBNF Guide](#)

NEXT : 2. KOR : Structrued Data from Text

...

Llm

Json

Llama Cpp

Llamaindex



Follow

Written by Abdullah Şamil Güser

11 Followers

More from Abdullah Şamil Güser



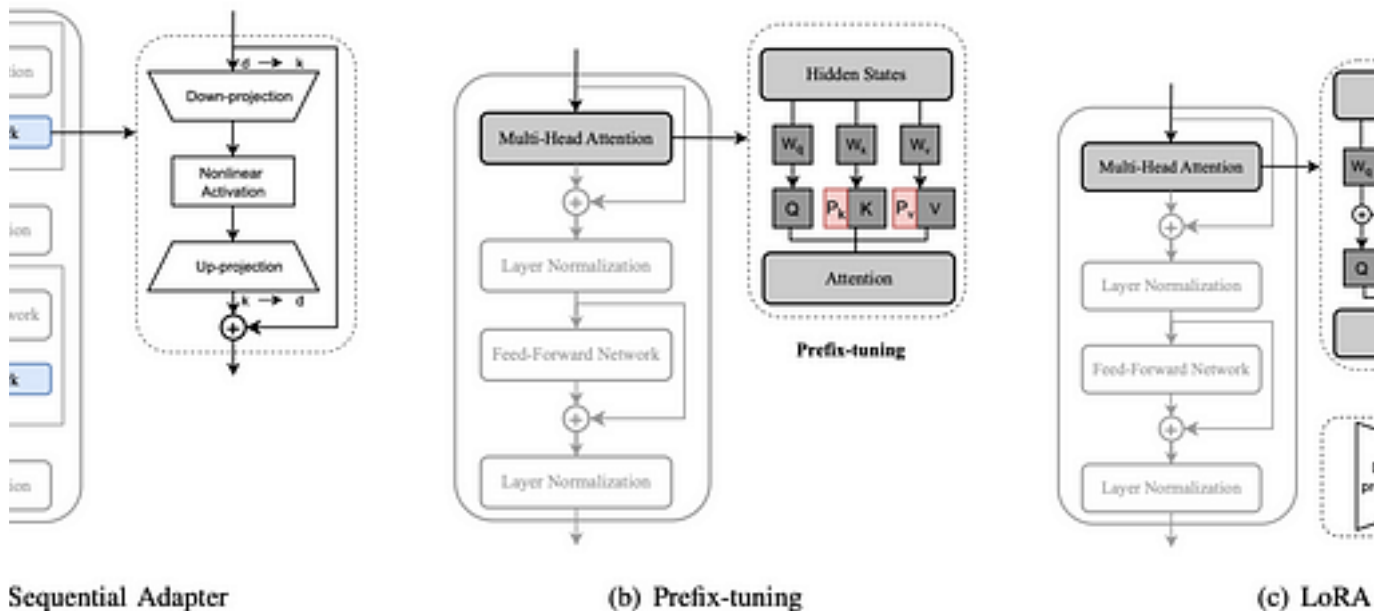
Abdullah Şamil Güser

Approximate Nearest Neighbor Methods

Explore Approximate Nearest Neighbor (ANN) methods, their importance, techniques like HNSW, LSH, ANNOY, Spill Trees, and Python libraries.

Dec 22, 2023 🖱 1



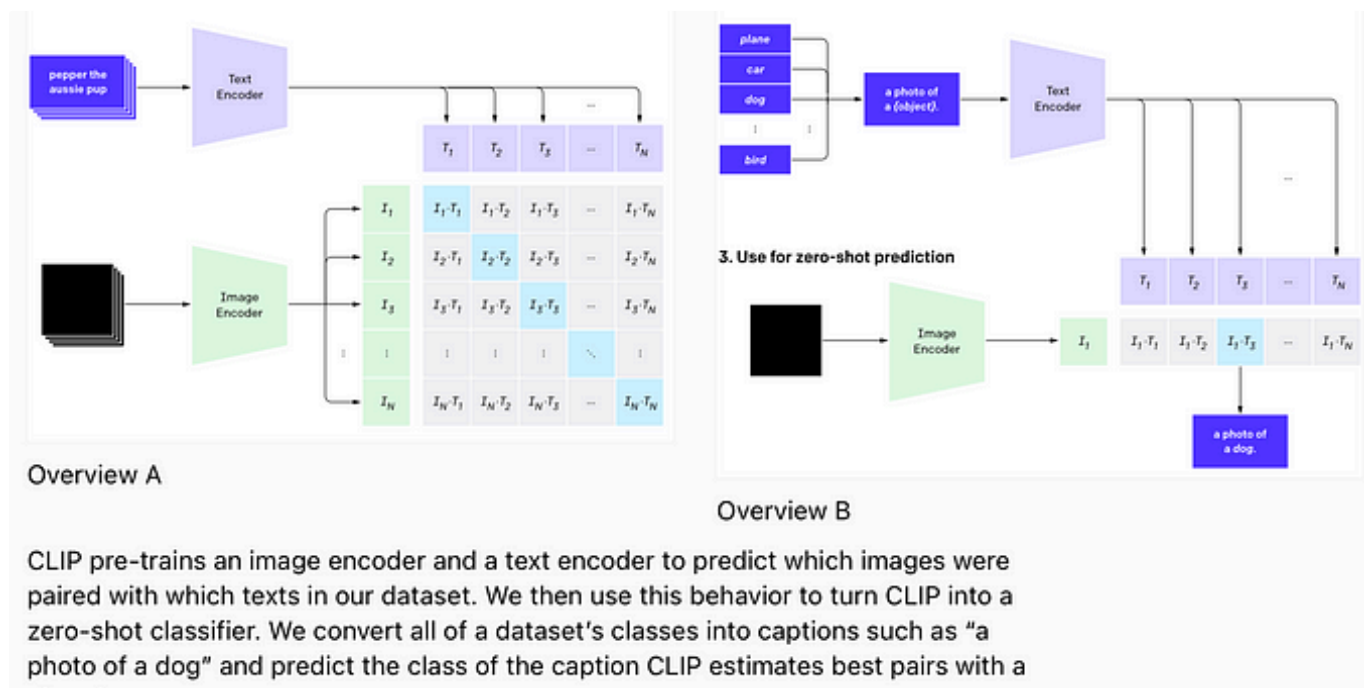


Abdullah Şamil Güser

Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and...

Summary of the Paper by Lingling Xu et. al.

Dec 29, 2023



Abdullah Şamil Güser

CLIP: Contrastive Language-Image Pretraining

Paper by Alec Radford, Ilya Sutskever et al. from OpenAI

May 5





 Abdullah Şamil Güser

Text Classification Using Class Information

How should our approach to text classification change if our classes also have meanings.

Dec 18, 2023  2

See all from Abdullah Şamil Güser

Recommended from Medium

	BENCHMARK	METRIC	Gemma 2		Llama 3		Grok-1
			9B	27B	8B	70B	314B
General	MMLU	5-shot, top-1	71.3	75.2	66.6	79.5	73.0
Reasoning	BBH	3-shot, CoT	68.2	74.9	61.1	81.3	–
	HellaSwag	10-shot	81.9	86.4	82	–	–
Math	GSM8K	5-shot, maj@1	68.6	74.0	45.7	–	62.9 (8-shot)
	MATH	4-shot	36.6	42.3	–	–	23.9
Code	HumanEval	pass@1	40.2	51.8	–	–	63.2 (0-shot)

 PinkHatHacker in AI Advances

Gemma 2 27B: The AI Already Better Than Chat GPT 4o

 Jun 29  139



 Dr. Walid Soula in Kinomoto.Mag AI

Fine-tuning LLama 3 with Unsloth

In this article I will show you how to fine-tune an LLM (Llama 3 from Meta) using Unsloth (including a way for custom dataset)

 Jun 4  88

Lists



Natural Language Processing

1592 stories · 1151 saves



ChatGPT prompts

48 stories · 1819 saves

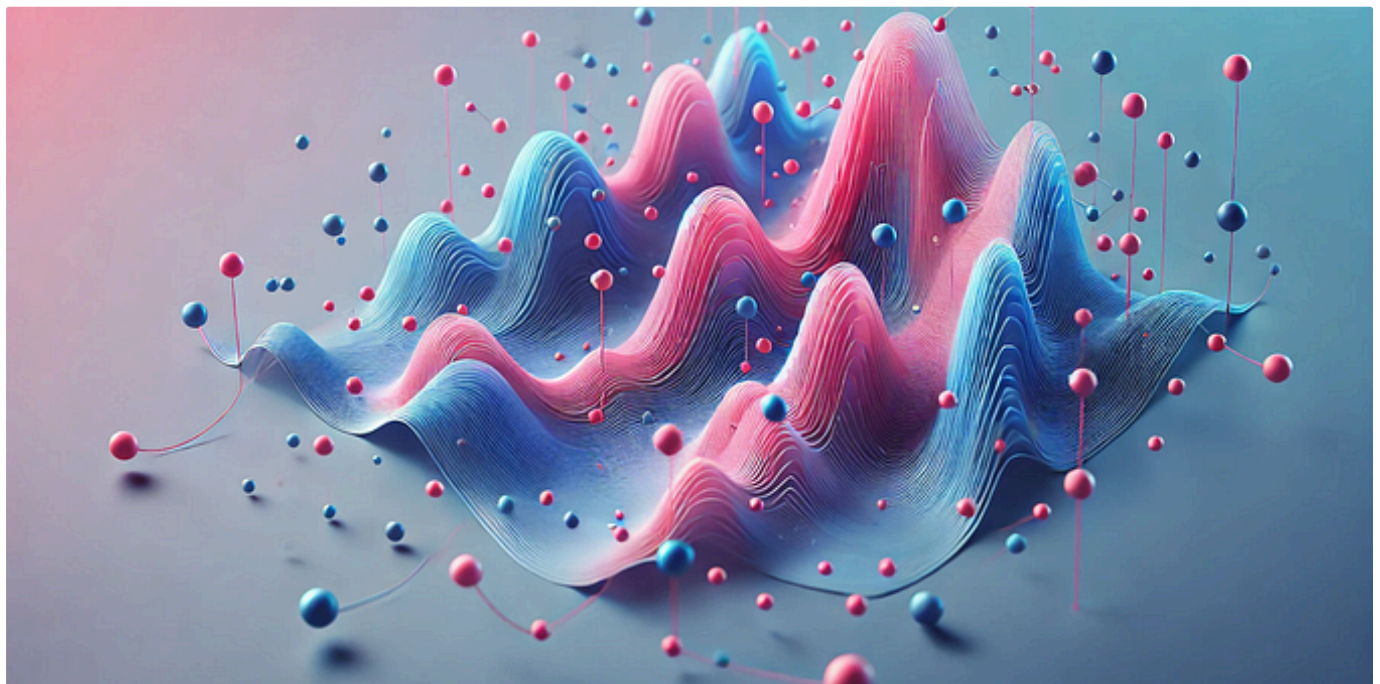


LEARN TO CODE

18 Advanced Python Libraries for Data Science You Need to Master

Python has become the go-to language for data science due to its simplicity, flexibility, and the vast number of libraries available. While...

★ Jul 14 🖱 35



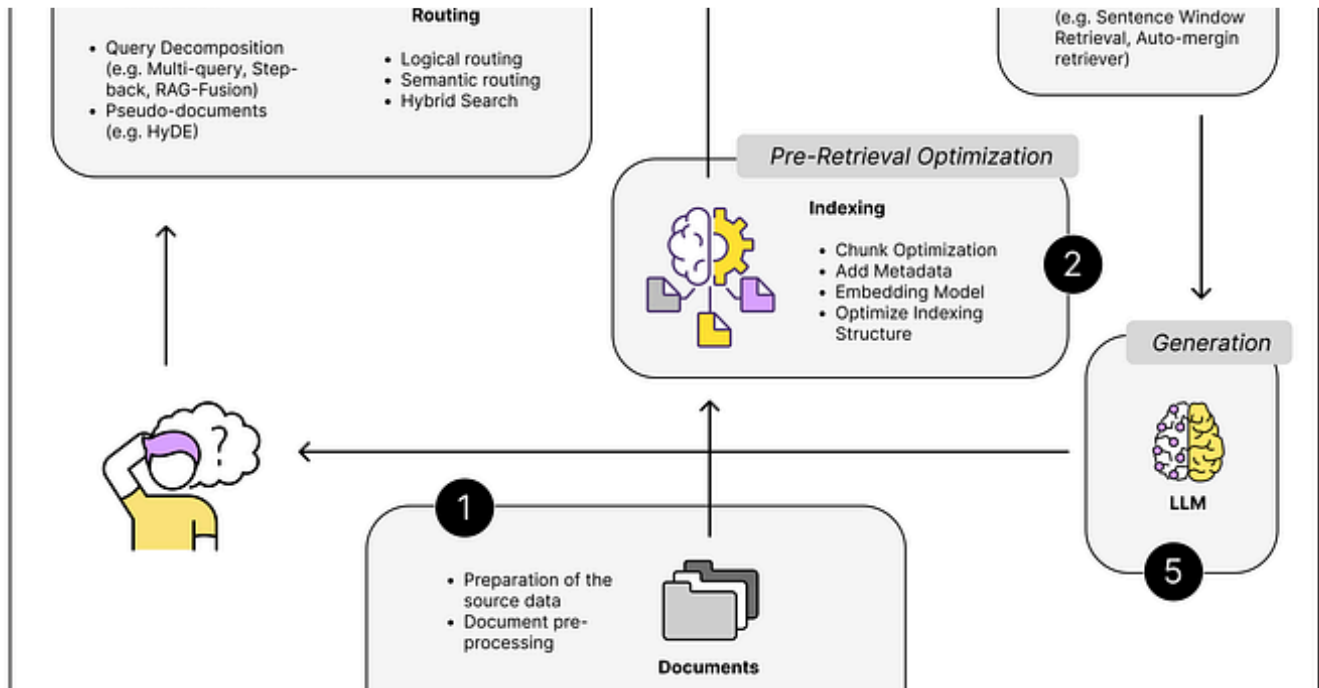
Thomas Reid in AI Advances

An introduction to Fabric: The best AI tool you've never heard of?

Does it wear well or will it crumple like a cheap suit

★ Jul 4 👤 738 💬 10

🔖 ⋮



👤 Dominik Polzer in Towards Data Science

17 (Advanced) RAG Techniques to Turn Your LLM App Prototype into a Production-Ready Solution

A collection of RAG techniques to help you develop your RAG app into something robust that will last

★ Jun 26 👤 1.99K 💬 20

🔖 ⋮



 Joshua Phuong Le in MITB For All

Extract Structured Output from Unstructured Texts with LLM Tool-Calling

1. INTRODUCTION

Jun 19  11

See more recommendations